Sustainable Resource Marketplace

A Technical Design Report

User Experience & Information Architecture • System Architecture • Technology Stack & Data Management

Prepared by the Office of the Chief Technology Officer
CTO Design Challenge Submission — July 2026

Table of Contents

Executive Summary 3

Deliverable I — UX & Information Architecture 3

1.1 Who Uses the Platform . . . 3

1.2 Information Architecture . . . 4

1.3 Core User Journey . . . 6

1.4 UX Principles . . . 6

Deliverable II — System Architecture 7

2.1 Architectural Approach . . . 7

2.2 Component Responsibilities . . . 8

2.3 Non-Functional Considerations . . . 8

Deliverable III — Technology Stack & Data 9

3.1 Recommended Technology Stack . . . 9

3.2 Hosting & Cloud Platform . . . 10

3.3 Core Data Model . . . 10

3.4 Data Organization Principles . . . 11

Closing: Roadmap & Trade-offs 12

Phased Roadmap . . . 12

Key Trade-offs & Assumptions . . . 12

Executive Summary

The Sustainable Resource Marketplace is a B2B platform that lets businesses list, discover, and exchange surplus or reusable materials — industrial byproducts, packaging, textiles, metal and wood offcuts, decommissioned equipment, and certified recycled inputs — instead of sending them to landfill or over-purchasing virgin resources. The platform's job is to make it as easy to source a verified sustainable resource as it is to buy from a conventional supplier, while making the environmental impact of every transaction visible and auditable.

This report sets out the technical direction across three areas: how users experience the product end-to-end, how the system is architected to support that experience reliably, and which technologies and data structures underpin it. Four guiding principles run through every decision below:

- **Trust is the product.** Business verification, transaction escrow, and transparent sustainability metrics are treated as core features, not add-ons — without them the marketplace has no credibility.
- **Sustainability data is first-class.** CO_2e avoided and waste diverted are computed and stored alongside every listing and transaction, not bolted on as a marketing report.
- **Modular before distributed.** The system starts as a well-factored modular monolith with clear service boundaries, so it can be split into independently deployed services only when real load demands it — avoiding premature infrastructure cost.
- **Buy commodity, build differentiation.** Payments, KYC, maps, and messaging infrastructure are bought from managed providers; engineering effort is spent on matching, sustainability scoring, and marketplace trust mechanics, which are the platform's actual differentiators.

Deliverable I — User Experience & Information Architecture

1.1 Who Uses the Platform

Three primary personas drive every design decision. Suppliers and buyers are frequently the same organization wearing different hats on different listings, so account roles are additive rather than exclusive.

Persona	Who they are	Primary goal	Key needs
Supplier (Seller)	Operations or sustainability manager at a business with surplus material, byproducts, or decommissioned assets.	Turn a liability (waste, storage cost, disposal fees) into revenue or a documented sustainability outcome.	Fast listing creation, credible buyer inquiries, simple negotiation, proof of pickup/delivery.
Buyer (Procurer)	Procurement or sustainability manager sourcing lower-cost or lower-impact material inputs.	Find verified, fit-for-purpose resources quickly and document the sustainability impact for internal/ESG reporting.	Reliable search and filtering, seller credibility signals, predictable logistics and pricing.

Persona	Who they are	Primary goal	Key needs
Platform Admin	Trust & safety / operations staff employed by the marketplace.	Keep the marketplace safe, resolve disputes, and keep verification and content quality high.	Moderation queues, dispute tooling, audit trails, aggregate analytics.

1.2 Information Architecture

The site is organized around six public-facing sections plus an isolated admin area. Public marketing and discovery content (Home, Marketplace browse, Company profiles, Resources) is reachable without an account so search engines and first-time visitors can evaluate the platform. Anything transactional (listing creation, dashboards, messaging, orders) sits behind an authenticated, KYC-verified business account. Admin tooling is not linked from the public navigation at all and is reached through a separate, role-gated console.

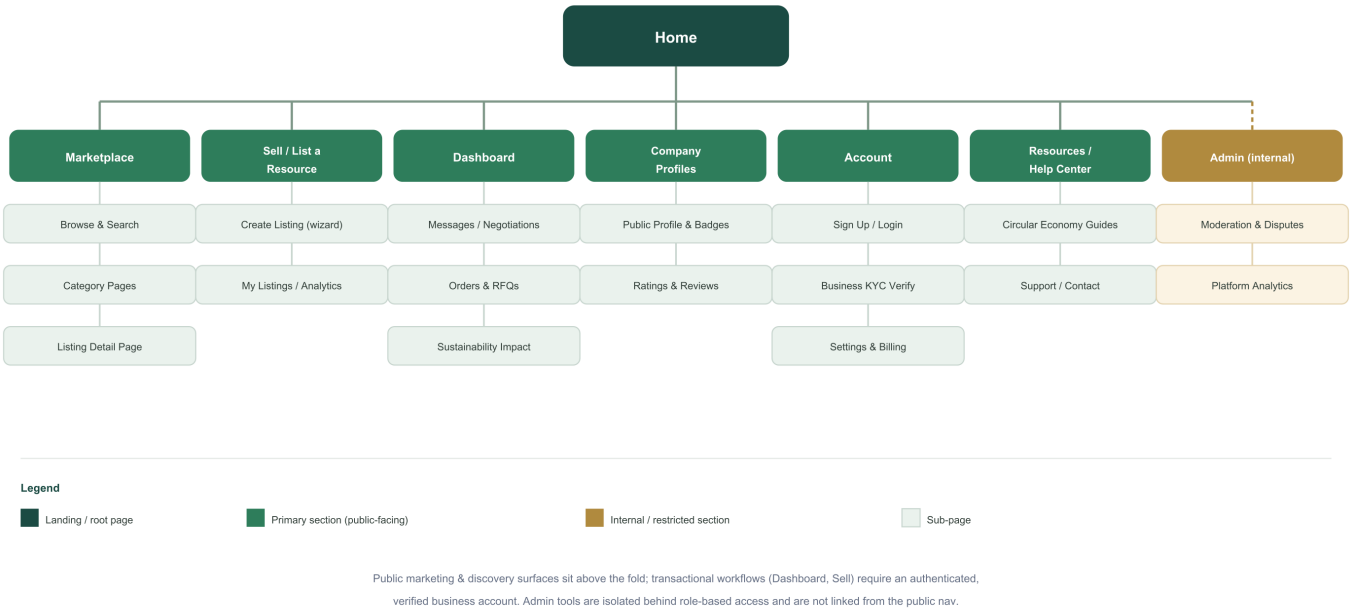


Figure 1. Site map / information architecture, showing public vs. authenticated vs. internal surfaces.

Page	Purpose
Home	Value proposition, category shortcuts, featured listings, aggregate impact stats (tonnes of waste diverted platform-wide) to build trust on arrival.

Page	Purpose
Marketplace (Browse & Search)	Faceted search over listings: category, location/radius, quantity, condition, exchange type (sell / donate / swap), certifications.
Listing Detail	Full specs, photos, seller profile summary, computed sustainability metrics (CO_2e avoided, waste diverted), and the primary RFQ / message CTA.
Create Listing (wizard)	Guided multi-step form: category & specs → quantity/condition → photos & documents → sustainability tagging → pricing/exchange terms.
Seller / Buyer Dashboard	Active listings or open RFQs, unread messages, orders in flight, and a running sustainability impact tally.
Messaging & Negotiation	Real-time thread per transaction; structured fields for offer price, quantity, and pickup window alongside free-text chat.
Transaction / Checkout	Reviews agreed terms, initiates escrow payment, and schedules logistics.
Order Tracking	Shipment status, carrier tracking, delivery confirmation, dispute entry point.
Company Profile	Verification badge, certifications, ratings, transaction history — the platform's core trust signal.
Sustainability Impact Report	Per-transaction and cumulative CO_2e / waste-diverted figures, exportable for ESG reporting.
Admin Console	Business verification queue, listing moderation, dispute resolution, platform analytics. Not linked from public nav; role-restricted.

1.3 Core User Journey

Buyers and suppliers move through parallel journeys that converge at a single shared transaction flow, then diverge again into shared post-transaction outcomes (reviews, reputation, and the impact report). Keeping transaction and fulfillment logic in one shared flow — rather than duplicating it per role — is what keeps escrow, logistics, and dispute handling consistent regardless of who initiated the deal.

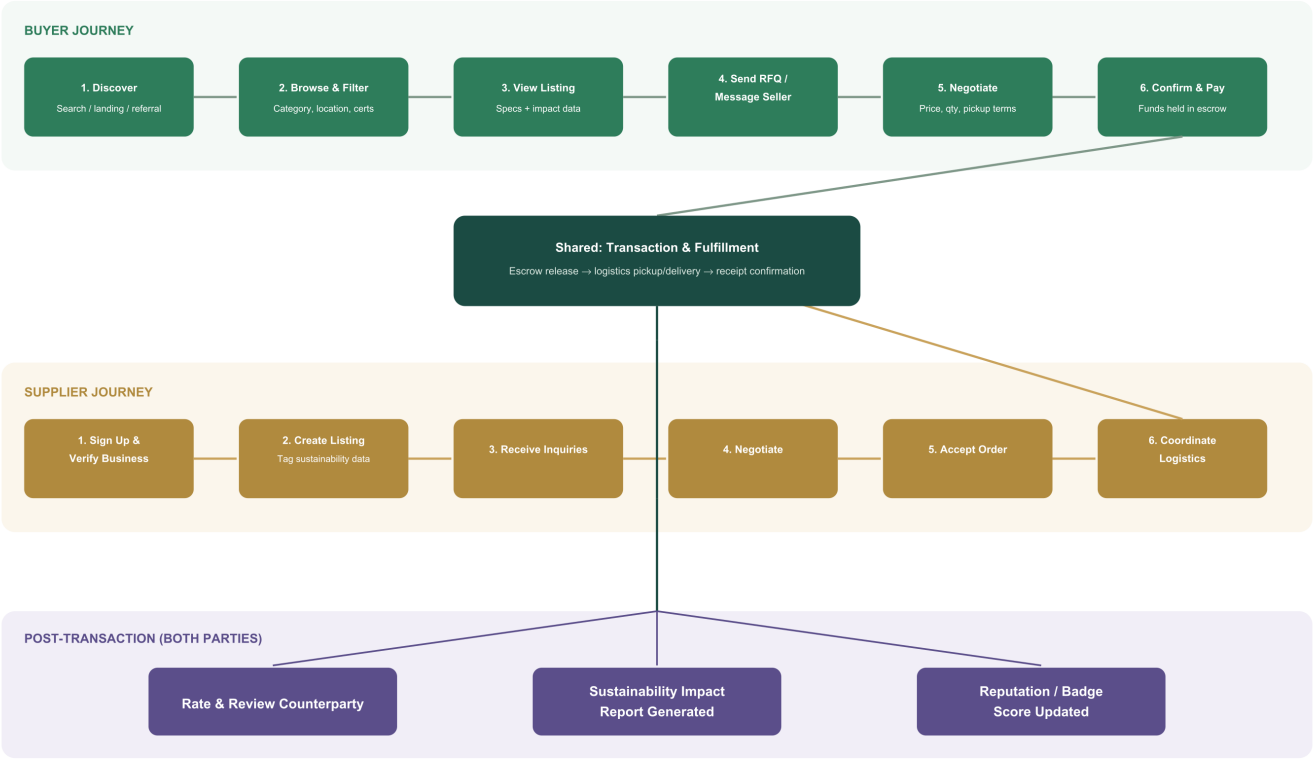


Figure 2. End-to-end buyer and supplier journeys, converging on a shared transaction and fulfillment flow.

1.4 UX Principles

- **Sustainability data up front, not buried.** Impact metrics appear on the listing card in search results, not just the detail page, so they factor into the buyer's decision from the first scan.
- **Low-friction listing creation.** The listing wizard pre-fills sustainability tags from category selection (e.g. “metal offcuts” auto-suggests a diversion-from-landfill metric) so suppliers aren't required to be sustainability experts to list correctly.
- **Trust signals everywhere a decision is made.** Verification badges, ratings, and response-time indicators appear on listing cards, profiles, and inside negotiation threads.
- **Mobile-responsive by default.** Warehouse and facilities staff who physically handle the material are frequently the ones photographing and listing it on-site; the listing wizard and camera capture must work well on a phone.

Deliverable II — System Architecture

2.1 Architectural Approach

The system is a modular monolith at launch: one deployable backend, organized internally into the same service boundaries it would have as microservices, communicating through well-defined interfaces and an internal event bus rather than shared database tables. This keeps operational overhead low for a small engineering team while preserving the seams needed to extract any module — Payments and Search are the most likely first candidates — into an independently scaled service once real traffic demands it. State changes (an RFQ is created, a payment is captured, a shipment updates) are published as events so that notification, matching, and analytics logic stay decoupled from the transactional path.

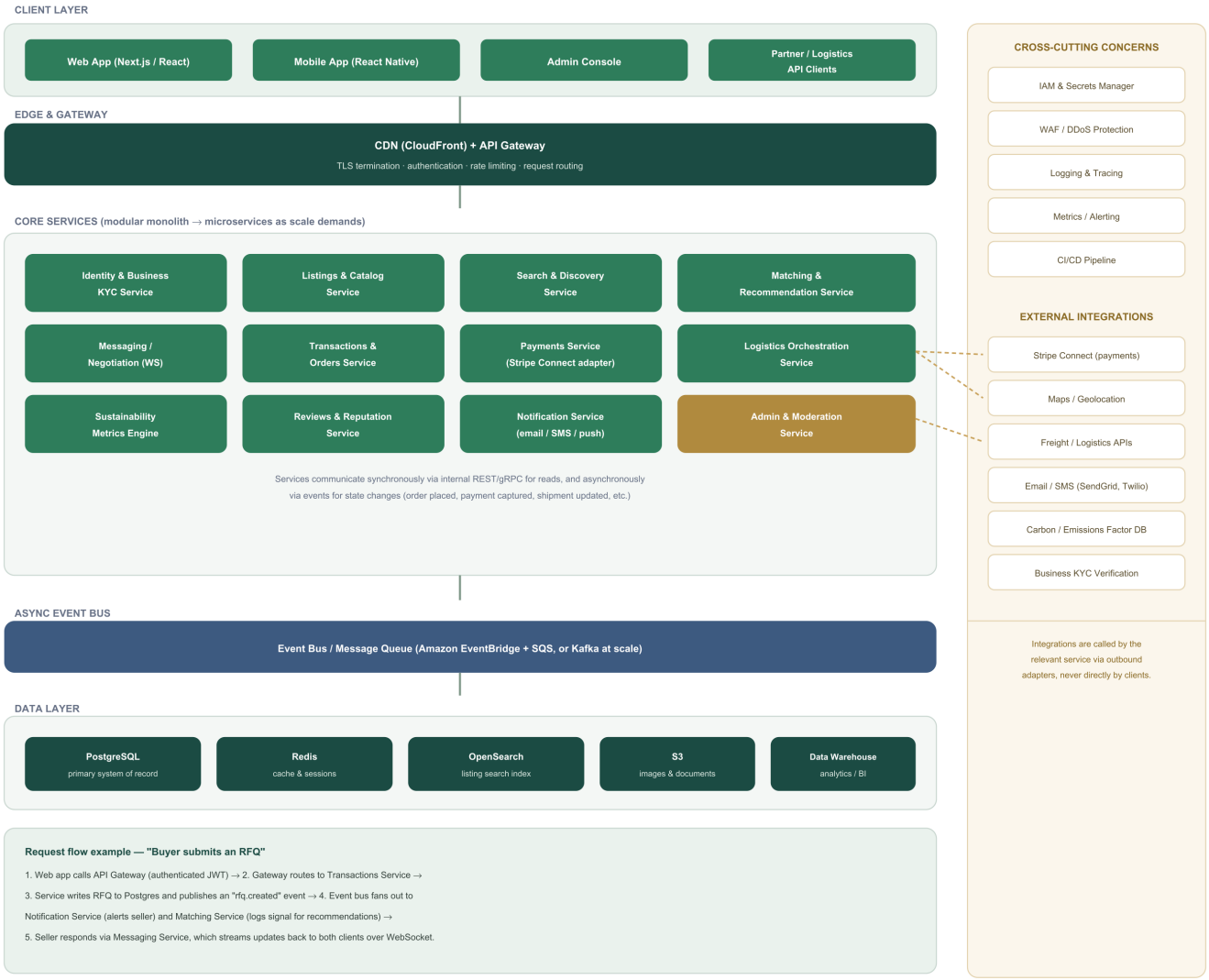


Figure 3. High-level system architecture: client layer, edge/gateway, core services, async event bus, data layer, and cross-cutting/external concerns.

2.2 Component Responsibilities

Component	Responsibility
API Gateway	TLS termination, JWT authentication, rate limiting, and routing to internal services; the single entry point for all client traffic.
Identity & Business KYC	Account auth, role management, and the business-verification workflow (document upload → third-party KYC check → verified badge).
Listings & Catalog	CRUD for listings, category taxonomy, media references.
Search & Discovery	Indexes listings into OpenSearch; serves faceted search and filtering.
Matching & Recommendation	Surfaces relevant listings to buyers and relevant buyers to new listings using category, location, and past-transaction signals.
Messaging / Negotiation	Real-time chat over WebSocket, with structured offer fields attached to each transaction thread.
Transactions & Orders	Owns transaction state machine (RFQ → negotiating → paid → fulfilled → closed) and coordinates the other services around it.
Payments	Thin adapter over Stripe Connect; holds funds in escrow and releases on delivery confirmation.
Logistics Orchestration	Coordinates pickup/delivery scheduling with third-party carrier APIs and tracks shipment status.
Sustainability Metrics Engine	Computes CO_2e-avoided and waste-diverted figures per listing and transaction from category-level conversion factors.
Reviews & Reputation	Post-transaction ratings and the aggregate reputation score shown on profiles.
Notification	Fans out events to email, SMS, and in-app push.
Admin & Moderation	Verification review queue, listing moderation, dispute resolution tooling.

2.3 Non-Functional Considerations

- **Scalability.** Stateless services behind the gateway scale horizontally; Search and Payments are isolated behind their own interfaces so they can be pulled out and scaled independently first.
- **Reliability.** Event-driven decoupling means a Notification-service outage never blocks a payment or an RFQ; the queue retries with a dead-letter path for manual triage.
- **Security.** Business KYC verification gates listing and purchasing; payments are handled entirely by Stripe Connect so the platform never touches raw card data (PCI scope stays minimal); secrets and IAM are centrally managed rather than embedded per-service.
- **Observability.** Centralized structured logging and distributed tracing across services, with alerting on transaction-flow error rates and payment/escrow anomalies specifically, since those are the highest-trust-cost failures.

Deliverable III — Technology Stack & Data Management

3.1 Recommended Technology Stack

Layer	Technology	Why
Frontend (web)	Next.js (React) + TypeScript, Tailwind CSS	Server-side rendering gives fast first paint and SEO for public listing/marketing pages, which matters for organic discovery; TypeScript keeps a large codebase maintainable across a growing team.
Mobile (future)	React Native	Shares component logic and the team's existing React/TypeScript skillset with the web app, minimizing the cost of a second client.
Backend services	Node.js (NestJS), TypeScript	TypeScript end-to-end reduces context-switching and lets frontend and backend share type definitions for API contracts; NestJS's module system maps cleanly onto the modular-monolith service boundaries.
Sustainability Metrics Engine	Python (FastAPI)	Impact calculations and future ML-based matching benefit from Python's data/ML ecosystem; isolating this as its own service keeps the core Node.js codebase uniform.
Realtime messaging	WebSocket via Socket.IO	Low-latency negotiation chat and live order-status updates without polling.
Async event bus	Amazon EventBridge + SQS (Kafka as a later scale option)	Fully managed, no cluster to operate at launch; SQS's dead-letter queues give free retry/failure handling. Kafka is a straightforward later swap if event volume or replay needs grow.
Primary database	PostgreSQL (Amazon RDS)	Strong relational integrity for transactions and payments, native JSONB for the variable attributes different resource categories need, and a mature managed offering.
Cache / sessions	Redis (Amazon ElastiCache)	Session storage, rate-limit counters, and hot-path caching (e.g. category trees, search facets).
Search index	OpenSearch	Faceted, typo-tolerant search over listings at a scale Postgres full-text search won't comfortably cover; AWS-native and open-source (avoids Elasticsearch licensing complications).
Object storage	Amazon S3 + CloudFront	Durable, cheap storage for listing photos, certification documents, and KYC files, served through a CDN for fast image loads globally.
Payments	Stripe Connect	Purpose-built for marketplaces: escrow-style holds, split payments between buyer/seller/platform fee, and PCI compliance handled by Stripe rather than us.
Auth & identity	Auth0 (or Amazon Cognito)	Managed auth with role-based access control, so engineering time goes into the business-KYC workflow rather than reimplementing session security.

Layer	Technology	Why
Hosting / compute	AWS ECS on Fargate	Containerized services without managing servers or a Kubernetes control plane; a natural upgrade path to EKS if the team later needs Kubernetes-specific tooling.
Infrastructure as code	Terraform	Reproducible environments and auditable infrastructure changes across dev/staging/production.
CI/CD	GitHub Actions	Tight integration with the source repo; sufficient for a small team without standing up separate CI infrastructure.
Observability	Datadog (or Grafana + Prometheus) and Sentry	Unified logs/metrics/traces plus dedicated error tracking, so incidents in the payment or transaction path surface immediately.
Analytics / BI	Segment → Snowflake or BigQuery → Metabase	Decouples product analytics and ESG-style aggregate reporting from the transactional database.

3.2 Hosting & Cloud Platform

AWS is the recommended cloud platform: it has the deepest bench of managed services used above (RDS, ElastiCache, OpenSearch Service, SQS/EventBridge, Fargate), reducing the operational surface a small platform team has to own directly. The same architecture maps onto GCP or Azure with equivalent managed services if the founding team has stronger existing expertise there — the service boundaries and data model in this report are cloud-agnostic by design.

3.3 Core Data Model

Each service owns its own tables and exposes data through its API rather than allowing cross-service joins, which keeps the modular-monolith boundaries real even though everything currently runs against one PostgreSQL instance. **Listing** and **Transaction** are the two central entities: a Listing belongs to an Organization and a Category and has exactly one computed SustainabilityMetric; a Transaction links a Listing to a buyer and seller Organization and fans out to Messages, a Payment, a Shipment, and Reviews.

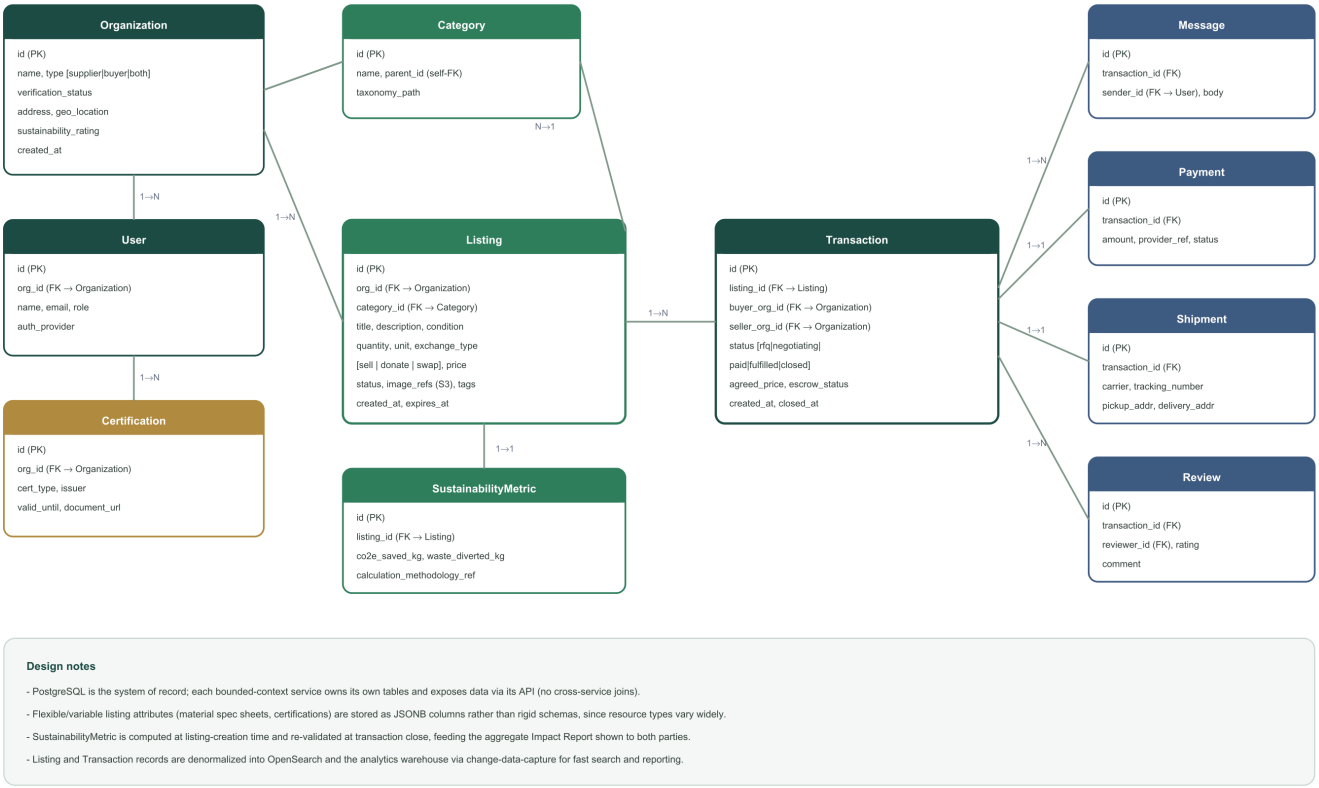


Figure 4. Core entity-relationship model.

3.4 Data Organization Principles

- **Flexible attributes via JSONB.** Resource categories vary too widely for a single rigid schema (a steel offcut and a pallet of packaging film have almost nothing in common); category-specific spec fields live in a JSONB column on Listing rather than as sparse dedicated columns.
- **Search and analytics are derived, not primary.** Listing and Transaction changes are streamed via change-data-capture into OpenSearch (for search) and the data warehouse (for BI/ESG reporting) — Postgres remains the single system of record.
- **Sustainability metrics are computed twice.** Once at listing creation (an estimate, for buyer decision-making) and once at transaction close (a validated figure, once actual quantity and condition are confirmed), so impact reports reflect what actually happened, not just what was listed.
- **Sensitive documents are access-scoped, not just stored.** KYC documents and certifications sit in a private S3 prefix with signed, time-limited URLs issued per request — never a public bucket.

Closing: Roadmap & Trade-offs

Phased Roadmap

Phase	Scope
MVP	Web app only; core marketplace (browse, list, RFQ, negotiate); Stripe Connect payments; manually coordinated logistics (platform facilitates, doesn't automate carrier booking); basic sustainability metric calculation.
V2	React Native mobile app; automated carrier/logistics API integration; recommendation engine using real transaction signal; expanded certification/verification partners.
V3	Enterprise ERP/procurement-system integrations (API access for large buyers); marketplace-wide sustainability intelligence and benchmarking; potential carbon-credit/offset marketplace extension.

Key Trade-offs & Assumptions

- A modular monolith is chosen over microservices at launch to prioritize engineering velocity for a small team; the explicit service boundaries and event-driven communication mean this is a deliberately reversible decision, not a dead end.
- Logistics is treated as an orchestration/integration problem (carrier APIs) rather than something the platform operates directly — owning trucks or warehouses is out of scope for this design.
- Sustainability metrics rely on category-level conversion factors (an accepted industry approach) rather than per-item lifecycle assessment, trading some precision for a launch-ready implementation.
- Payments assume a commission or subscription revenue model compatible with Stripe Connect's split-payment support; the specific monetization model is a product decision outside this report's scope.

This report intentionally favors clarity and defensible reasoning over exhaustive implementation detail, per the challenge's evaluation focus: a logical, well-justified design that a team could start building from directly.